24BCE5063 | Siddhartha Gupta

Compiler Design

Lab Assignment 7 – Shift Reduce Parser

AIM

To design and implement a **Shift-Reduce Parser** in C that analyzes an input string according to a given context-free grammar using the **shift** and **reduce** operations.

```
#include <stdio.h>
#include <string.h>

#define MAX 20

char lhs[MAX], rhs[MAX][MAX];
int n;

char stack[MAX], input[MAX];
int top = -1;

void display(int pos, char *action)
{
    int i;

    printf("$");

    for (i = 0; i <= top; i++)
        printf("%c", stack[i]);

    printf("\t");

    for (i = pos; input[i] != '\0'; i++)
        printf("%c", input[i]);

    printf("\t\t%s\n", action);
}

int reduce()
{
    int i, len, start;

    for (i = 0; i < n; i++)
    {
        len = strlen(rhs[i]);

        if (top + 1 >= len)
        {
            start = top - len + 1;
            if (strncmp(&stack[start], rhs[i], len) == 0)
            {
                top = start - 1;
                stack[++top] = lhs[i];
```

```
                printf("$");

                for (int j = 0; j <= top; j++)
                    printf("%c", stack[j]);

                printf("\t\tREDUCE %c -> %s\n",
                       lhs[i], rhs[i]);

                return 1;
            }
        }
    }

    return 0;
}

int main()
{
    int i, pos = 0;
    char action[30];

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("\nEnter productions without spaces:\n");
    printf("Example: S -> aA | b\n\n");
    for (i = 0; i < n; i++)
    {
        printf("Production %d: ", i + 1);
        scanf("%s %s", &lhs[i], rhs[i]);
    }

    printf("\nEnter input string: ");
    scanf("%s", input);

    printf("\nGRAMMAR:\n");

    for (i = 0; i < n; i++)
        printf("%c -> %s\n", lhs[i], rhs[i]);

    printf("\nSTACK\t\tINPUT\t\tACTION\n");
    printf("$\t\t%s\t\tSTART\n", input);

    while (input[pos] != '\0')
    {
        stack[++top] = input[pos];

        sprintf(action, "SHIFT %c", input[pos]);
        pos++;

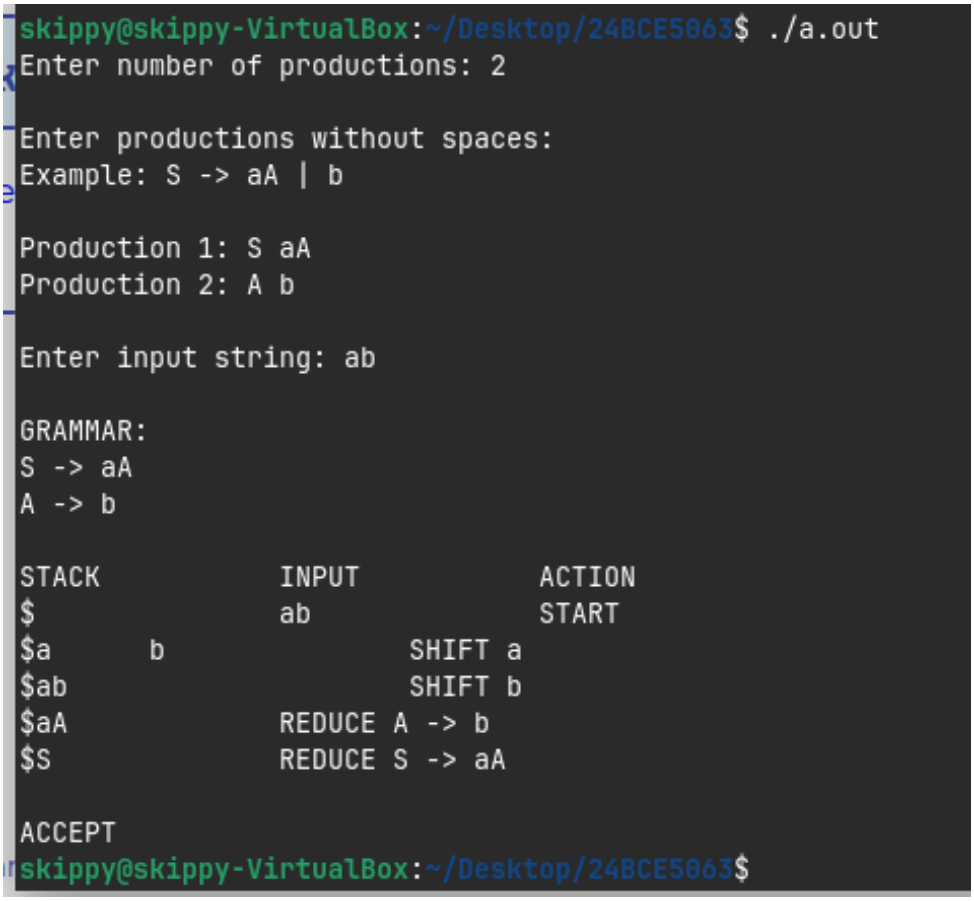
        display(pos, action);
        while (reduce())
        {
        }
    }
```

```
    while (reduce())
    {
    }

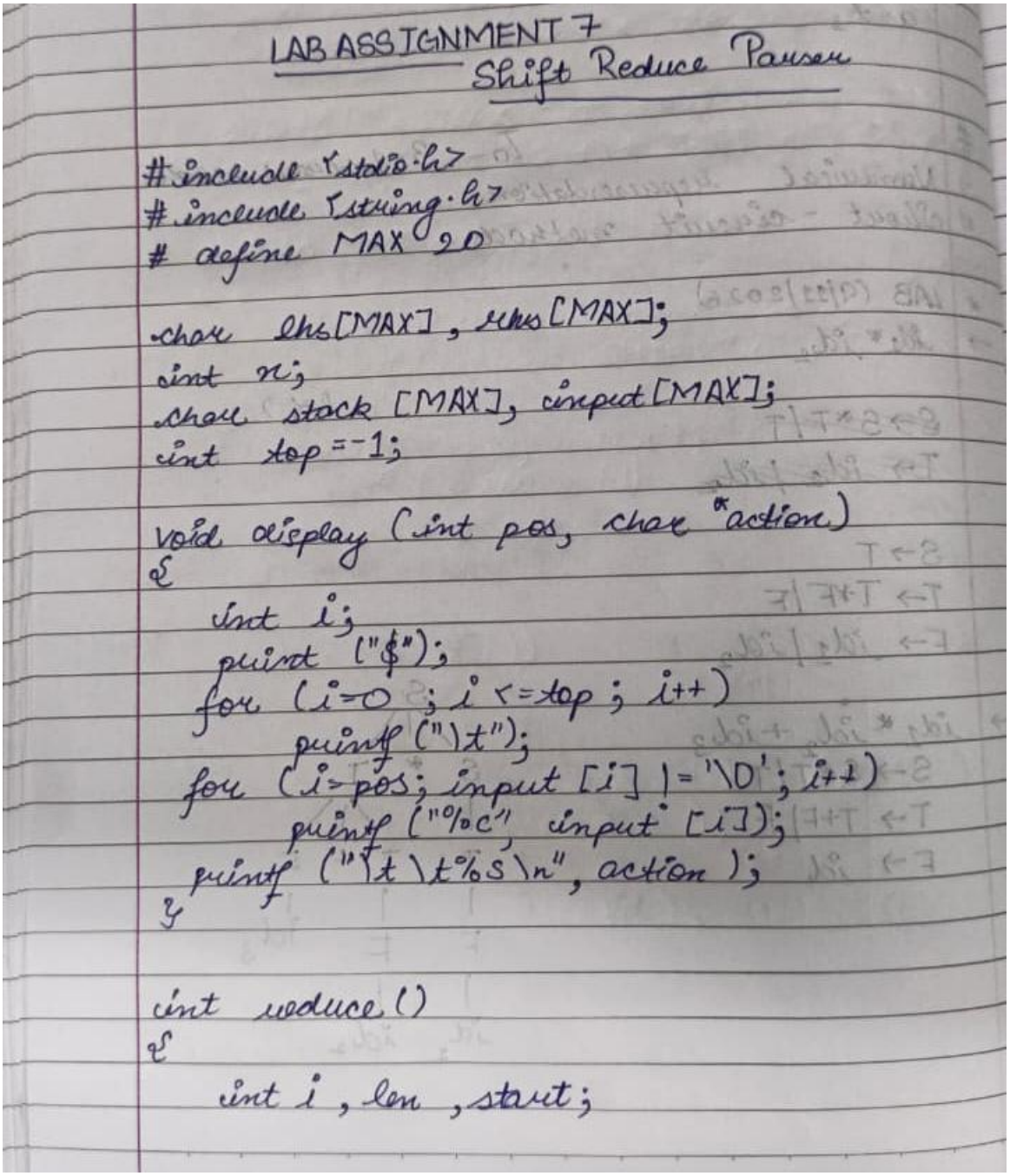
    if (top == 0 && stack[0] == lhs[0])
        printf("\nACCEPT\n");
    else
        printf("\nREJECT\n");

    return 0;
}
```

OUTPUT



1



LAB ASSIGNMENT 7
Shift Reduce Parser

```
#include <stdio.h>
#include <string.h>
# define MAX 20

char lhs[MAX], rhs[MAX];
int n;
char stack [MAX], input[MAX];
int top = -1;

void display (int pos, char *action)
{
    int i;
    printf ("$");
    for (i=0 ; i <= top ; i++)
        printf ("\t");
    for (i=pos; input [i] != '\0'; i++)
        printf ("%c", input [i]);
    printf ("\t\t%s\n", action );
}

int reduce ()
{
    int i , len , start;
```

2

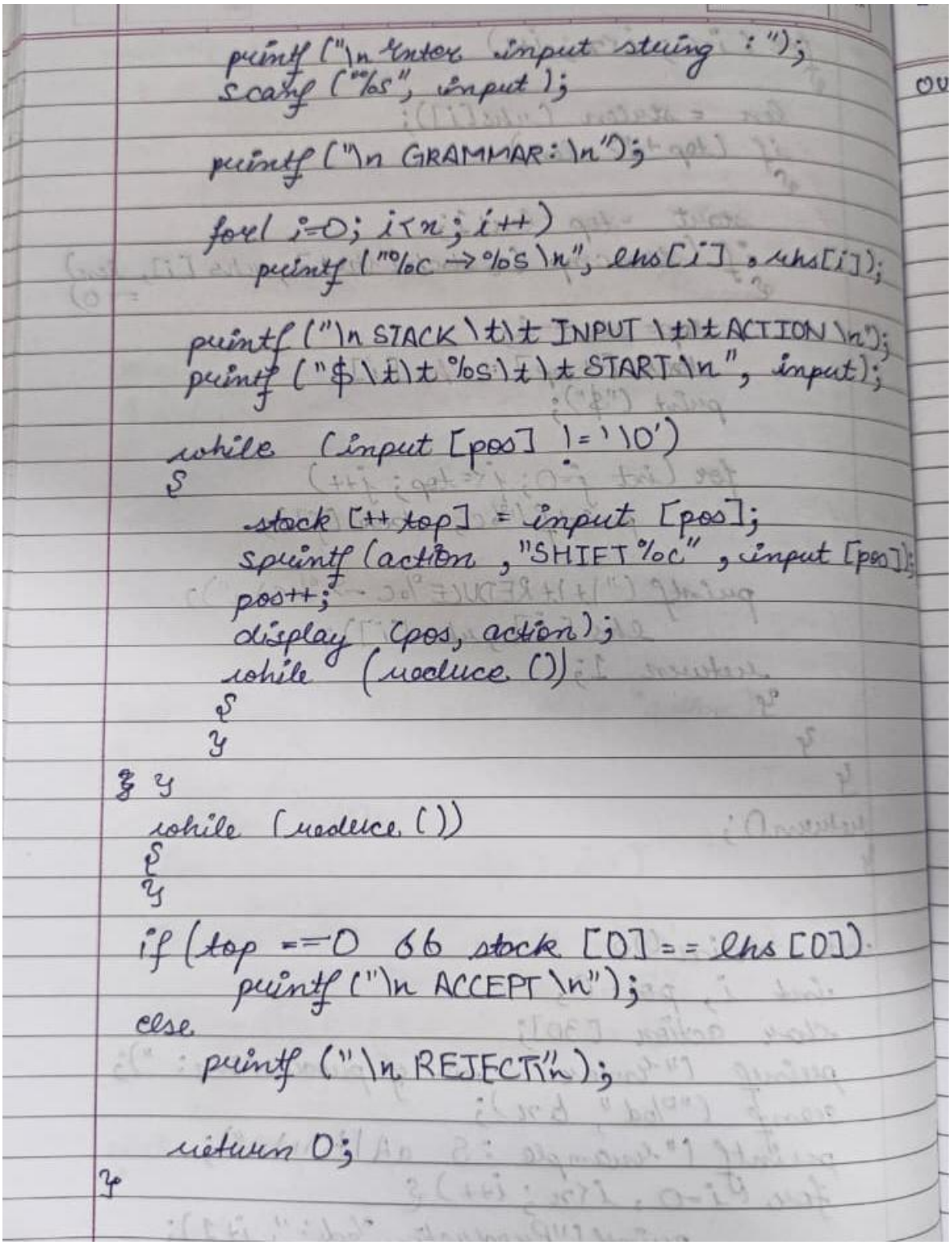
```
for (i=0; i<n; i++)
{
  len = strlen (rhs[i]);
  if (top+1>=len)
  {
    start = top - len+1;
    if (strncmp (&stack[start], rhs[i], len)
                                    == 0)
    {
      top= start-1;
      stack [++top] = lhs [i];
      print ("$");

      for (int j=0; j<=top; j++)
          printf ("%c", stack [j]);

      printf ("\t\tREDUCE %c -> %s\n",
            lhs [i], rhs [i]);
      return 1;
    }
  }
}
return 0;
}

int main ()
{
  int i, pos=0;
  char action [30];
  printf ("Enter no. of productions: ");
  scanf ("%d", &n);
  printf ("Example : S aA|b\n\n");
  for (i=0, i<n; i++) {
        printf ("Production %d: ", i+1);
        scanf ("%s%s", &lhs [i], rhs [i]);
  }
```

3



```
printf("\n Enter input string : ");
scanf("%s", input);

printf("\n GRAMMAR:\n");

for(i=0; i<n; i++)
    printf("%c -> %s\n", lhs[i], rhs[i]);

printf("\n STACK \t\t INPUT \t\t ACTION \n");
printf("$ \t\t %s \t\t START \n", input);

while (input[pos] != '\0')
{
    stack[++top] = input[pos];
    sprintf(action, "SHIFT %c", input[pos]);
    pos++;
    display(pos, action);
    while (reduce())
    {
    }
}
while (reduce())
{
}
if (top == 0 && stack[0] == lhs[0])
    printf("\n ACCEPT \n");
else
    printf("\n REJECT \n");

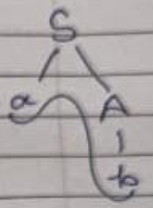
return 0;
}
```

4

INPUT: Enter number of productions : 2
Production 1: S aA
Production 2: Ab
Enter input string : ab

OUTPUT: GRAMMAR
S → aA
A → b

STACK	INPUT	ACTION
$	ab	START
$ a	b	SHIFT a
$ ab		SHIFT b
$ aA	→ b	REDUCE
$ S	→ aA	REDUCE
		Accept!



(Q1) Grammar ~~E → aEb~~ ~~E → ε~~
S → aA
A → b

Input → aa

STACK	INPUT	ACTION
$	aa	START
$ a	a	SHIFT a
$ aa		SHIFT a
		REject!

(Q2) S → AB
A → a
Input → aaab

STACK	INPUT	ACTION
$	aaab	START
$ a	aab	SHIFT a
$ aa	ab	SHIFT a
$ aaa	b	SHIFT a
$ aaab		SHIFT b
$ aaaS		REDUCE S → b
$ aaS		REDUCE S → aS
$ aS		REDUCE S → aS
$ S		REDUCE S → aS Accept

INFERENCE

The **shift-reduce parsing technique** was successfully studied and implemented using a stack-based C program. The parser performs **shift** operations to move input symbols onto the stack and **reduce** operations to replace recognized RHS patterns with their corresponding LHS non-terminals. The FIRST and FOLLOW sets for the given grammars were also calculated, which are useful in understanding and constructing bottom-up/top-down parsing mechanisms.